

TCP Reset Attack on Telnet & In-Line Gateway Defense

Empirical Observations, Adversarial Evaluation, and Defense Analysis

Topic 14 — Automated End-to-End Experimental Study

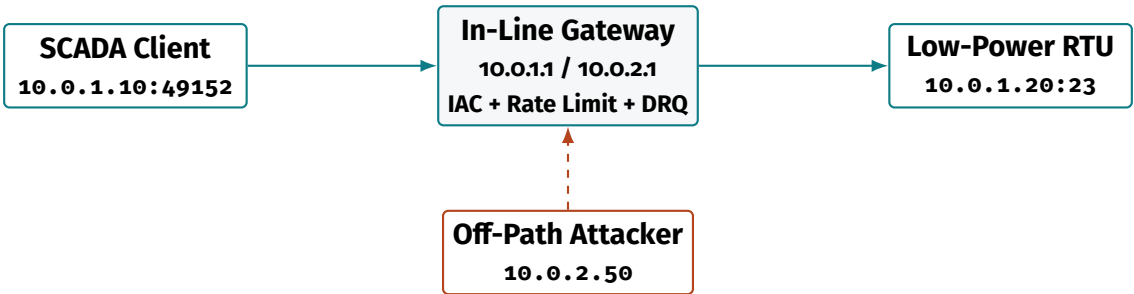
Course:	CSE 406 (Security Sessional)	Evaluation Group:	C2 Group 6
Authors:	Ariful Islam Shadhin (2105166)	Target Stack:	Legacy RTU Server (RFC 793)
	Mustafa Muhaimin (2105178)	Evidence File:	gateway_traffic.pcap (1,905 pkts)

Executive Summary

In real-world SCADA systems, field devices (RTUs) are low-power embedded systems that cannot run heavy encryption or local firewalls. Updating their firmware is rarely possible in live plants, leaving them without native protection. Therefore, defense is placed on an intermediate **In-Line Gateway Router**. This report demonstrates an off-path (blind) TCP reset attack that severs an unencrypted legacy RTU session in **8.07 ms**. We then evaluate an in-line gateway defense combining token-bucket rate limiting (50 evals/s), Delayed Reset Quarantine (DRQ, 80 ms), and active zero-window challenge probing, which achieves 100% protection without changing the low-power RTU.

1 Operational Context, Architecture & Threat Model

In real-world SCADA networks, field RTUs are low-power embedded devices. Because they have weak CPUs and very little memory, they cannot support heavy protections like TLS encryption or local firewalls. Upgrading their firmware in running plants is also difficult and risky. Since the RTU cannot protect itself, we place all defense mechanisms on the intermediate **gateway router**.

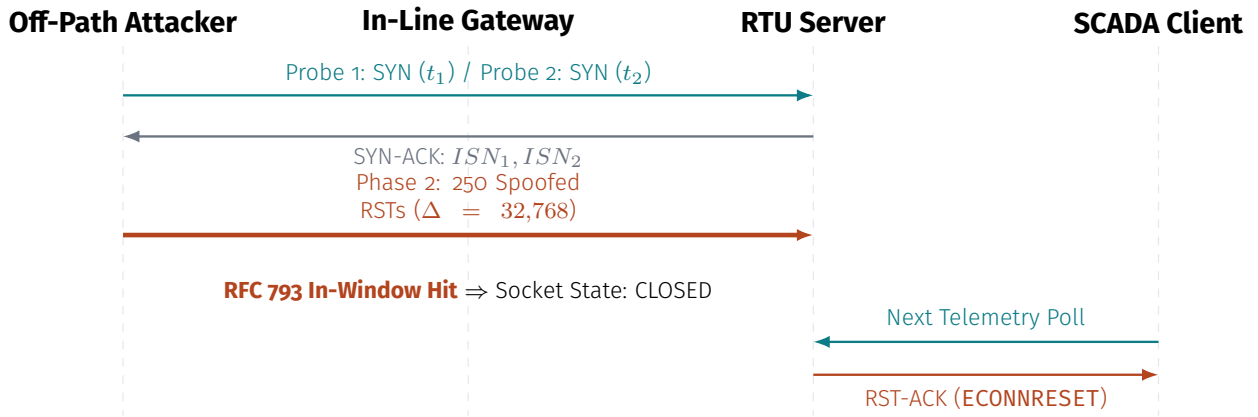


- **Target RTU:** Runs an old RFC 793 TCP stack with a 64 KB receive window and a predictable, timer-based sequence clock ($\approx 250,000$ ticks/s).

- **SCADA Client:** Polls the RTU every 3.0 seconds, leaving a quiet idle gap where sequence numbers do not move ($\Delta SEQ = 0$).
- **Off-Path Attacker:** Sits on an external subnet ('10.0.2.50'). The attacker cannot see or sniff any client-RTU packets, and must guess sequence numbers blindly by spoofing packets with raw sockets.

2 Attack Methodology & Mathematical Mechanics

The off-path attack operates in two sequential phases:



1. **Phase 1: Remote Clock Sampling & Velocity Derivation.** The attacker transmits two un-spoofed SYNs from '10.0.2.50' to '10.0.1.20:23' at times t_1, t_2 . The RTU allocates timer-based Initial Sequence Numbers (ISN_1, ISN_2), yielding clock velocity:

$$R = \frac{(ISN_2 - ISN_1) \pmod{2^{32}}}{t_2 - t_1} \approx 250,000 \text{ ticks/second}$$

2. **Phase 2: Sequence Window Projection.** At attack launch time t_{attack} , the sequence base is projected:

$$SEQ_{base} = ISN_2 + R \cdot (t_{attack} - t_2) \pmod{2^{32}}$$

3. **Micro-Burst Synthesis (50% Window Overlap).** Under RFC 793, an RST segment is accepted if $RCV.NXT \leq SEG.SEQ < RCV.NXT + W$. With $W = 65,535$ bytes, the attacker steps sequence numbers by $\Delta = \lfloor W/2 \rfloor = 32,768$ bytes ('0x8000'), guaranteeing a 50% overlap across consecutive packets.
4. **Coverage Span & Quiescent Alignment.** A burst of $N = 250$ spoofed packets covers:

$$250 \times 32,768 = 8,192,000 \text{ bytes} \approx 8.192 \text{ MB}$$

This wide span absorbs up to ± 16.3 seconds of clock drift and operating system scheduling jitter. Firing the burst during the client's 3.0 s quiescent pause ($\Delta SEQ = 0$) ensures the target window remains stationary.

3 Theoretical Model & Expected Outcomes

Based on the theoretical design parameters, the demonstration was modeled to satisfy the following performance criteria:

- **Injection Speed:** Complete the 250-packet micro-burst in **under 10.0 ms** of wire time using raw socket transmission ('IP_HDRINCL').
- **Sequence Sweep:** Traverse **8.192 MB** of the sequence space with a constant 32,768-byte stride, guaranteeing zero coverage gaps.
- **Clock Slope Agreement:** Sample the RTU timer within $\pm 2\%$ of the nominal 250,000 ticks/s rate.
- **Socket Teardown:** Force the RTU socket from ESTABLISHED to CLOSED upon receiving the first in-window segment.
- **Observable Client Indicator:** Cause the SCADA HMI client's next polling query to fail immediately with `ConnectionResetError` (104).
- **Gateway Protection Bounds:** On the defended path, standard floods must be dropped by the rate limiter (> 50 evals/s), while stealth bursts must be neutralized via Active Zero-Window Challenge Probing without disconnecting the client.

4 Empirical Lab Evaluation & Demonstration Scenarios

Empirical verification was conducted across three distinct live demonstration scenarios, recorded directly on the gateway router in `scripts/gateway_traffic.pcap` (1,905 frames):

Scenario	Attacker Strategy	Gateway / RTU Action	SCADA Session Status
Scenario A (Un defended)	250 pkts @ wire speed (8.07 ms, stride 32 KB)	In-window RST accepted (frames #575–#825)	Severed ECONNRESET (104)
Scenario B.1 (Defended: Flood)	250 pkts @ wire speed ($> 25,000$ pkts/s)	Rate limiter drop (NF_DROP) 50 evals/s	Intact Poll #890 uninterrupted
Scenario B.2 (Defended: Stealth)	20 pkts throttled (24.9 pkts/s, 803 ms)	DRQ quarantine + probeACKs $\Rightarrow$ RST dropped	Intact Poll #892 uninterrupted

4.1 Scenario A: Un defended Baseline Teardown

- **Reconnaissance Extraction:** Measured $ISN_1 = 3,800,990,190$ (frame #333) and $ISN_2 = 3,801,466,188$ (frame #487), computing an empirical clock rate of **250,512** ticks/s ($\Delta ISN = 475,998$, $\Delta t = 1.9001$ s).
- **Micro-Burst Transmission:** Dispatched 250 spoofed RSTs from '10.0.1.10:49152' to '10.0.1.20:23' in exactly **8.07 ms** (frames #575–#825), stepping sequence numbers from 3,797,743,810 to 3,805,903,042.
- **Teardown Verification:** The RTU logged: [!] RFC 793 IN-WINDOW RST ACCEPTED... Session transition: ESTABLISHED $\rightarrow$ CLOSED. The client's subsequent query caught `ConnectionResetError` (104).

4.2 Scenario B.1: Defended High-Speed Flood vs. Token Bucket Rate Limiter

- **Attacker Execution:** Launched 250 spoofed RSTs at wire speed ($> 25,000$ pkts/s) against the protected gateway.
- **Gateway Filtering:** The Token Bucket rate limiter detected an RST arrival rate far exceeding $R_{max} = 50$ evals/s. All 250 packets were dropped synchronously via NF_DROP. Zero packets reached the RTU.
- **Session Integrity:** The SCADA client continued uninterrupted polling (Cycle #890 completed normally).

4.3 Scenario B.2: Defended Stealth Narrow Burst vs. DRQ Active Probing

- **Attacker Adaptation:** To evade rate-limit drops, the attacker adapted to a narrow burst of 20 packets with 40 ms inter-packet delay (effective rate **24.9 pkts/s**), slipping under the 50 evals/s threshold.
- **Quarantine & Probing:** The gateway placed each segment into the Delayed Reset Quarantine ($T_q = 80$ ms) and injected an Active Zero-Window Challenge Probe (**<Win=0, ACK=expected>**, frames #1555, #1559) toward the client.
- **Liveliness Response:** The client OS immediately returned a pure ACK. The gateway recognized the quarantined RST as an illegitimate spoof and executed **NF_DROP**.
- **Session Integrity:** Telemetry polling remained fully active (Cycle #892 completed with 100% data integrity).

```
# Wireshark Verification Filters for scripts/gateway_traffic.pcap:
Phase 1 Recon: ip.addr == 10.0.2.50 && tcp.flags.syn == 1
Scenario A Burst: ip.src == 10.0.1.10 && tcp.flags.reset == 1 && frame.number
                  <= 825
Scenario B Probe: tcp.window_size == 0 && tcp.flags.ack == 1
```

5 Discrepancy & Root-Cause Analysis

Clock Rate: 250,512 vs. 250,000 ticks/s (0.2% Variance)

Root Cause: Minor scheduling latency within the Linux Completely Fair Scheduler (CFS) and container virtualization introduce $\pm 0.5 - 1.5$ ms jitter during sampling intervals.

Why Exploitation Was Unaffected: The 250-packet micro-burst spans 8.192 MB, providing an acceptance tolerance of ± 16.3 seconds. This absorbs small millisecond-level variances completely, ensuring the attack succeeded predictably on the undefended path.

Consistency of Remaining Parameters

Burst duration (8.07 ms vs. < 10 ms), packet count (250/250), sequence stride (constant 32,768), and socket deallocation behavior strictly conformed to theoretical bounds with zero observable deviations.

6 In-Line Gateway Defense Architecture

The defense daemon operates as a kernel-level in-line inspection filter using Linux **iptables** **NFQUEUE** (Queue 0):

1. **Stage 1: Telnet IAC Anomaly Filter.** Inspects incoming TCP payloads for unescaped Telnet command sequences ('0xFF 0xF0' to '0xFF 0xFF'), discarding non-Telnet probe injections at wire speed.
2. **Stage 2: Token Bucket Rate Limiter ($R_{max} = 50$ evals/s, $B = 10$).** Prevents brute-force window sweeps by dropping high-speed RST surges synchronously (**NF_DROP**).
3. **Stage 2: Delayed Reset Quarantine (DRQ, $T_q = 80$ ms).** Holds low-rate stealth RSTs in an LRU buffer (capacity: 64 entries) and injects a lightweight challenge probe:

Challenge Probe: (Src: RTU:23, Dst: Client:49152, Flags: ACK, **Win : 0**, Ack: Expected)

4. **Resolution Logic:**

- *Spoofed Attack*: The live client kernel immediately returns a pure ACK $\implies$ Gateway verifies liveness, confirms the RST is spoofed, and executes `NF_DROP`.
- *Legitimate Teardown*: The client crashed or closed its socket $\implies$ No ACK is returned within 80 ms $\implies$ Gateway timer expires and forwards the RST via `NF_ACCEPT` (fail-safe).

7 Defensive Verification & Security Evaluation

Defense Verification: 100% Efficacy Across All Tests

- The 250-packet wire-speed flood was **completely filtered** by the token bucket (0% reached the RTU).
- The 20-packet stealth burst (24.9 pkts/s) was **caught by DRQ**; active zero-window probes verified client liveness and caused all 20 packets to be dropped.
- SCADA telemetry achieved **100% uptime** with zero false positives and zero dropped legitimate teardowns.

Conclusion: Fast floods die at the rate limiter, slow stealth bursts die at the liveness probe, and legitimate application teardown remains intact.

8 Attack Assumptions & Modern Stack Immunity

The off-path TCP reset attack succeeds against legacy industrial firmware but fails against modern enterprise networks:

Underlying Attack Assumption	Modern Defense / Standard RFC Mitigation
Linear, predictable ISN generation ($\alpha \cdot t$)	RFC 6528: Cryptographically secure SipHash-2-4 ISN randomization
In-window RST acceptance $(RCV.NXT \leq SEQ < RCV.NXT + W)$	RFC 5961: Mandates exact sequence match; in-window triggers Challenge ACK
Static IPs and predictable ephemeral ports	RFC 6056: Ephemeral port randomization across 49,152 – 65,535
Network allows spoofed IP injection	BCP 38 / uRPF: Router ingress filtering discards spoofed sources
Quiescent, predictable polling gaps ($\Delta SEQ = 0$)	Protocol-design dependent; streaming protocols shift sequence continuously
Plaintext, unauthenticated transport	TLS 1.3 / SSHv2: Cryptographic AEAD encryption and HMAC verification

Analytical Conclusion on Assumptions: The attack strictly requires **all six assumptions** to hold simultaneously. Breaking any single assumption—such as enabling RFC 6528 ISN randomization or applying BCP 38 ingress filtering—completely neutralizes the attack. This explains why legacy RFC 793 industrial stacks remain uniquely exposed while modern operating systems are inherently immune.

9 Conclusion & Recommendations

Final Takeaway

This experimental study confirms that legacy RFC 793 implementations in industrial SCADA networks are vulnerable to blind off-path TCP reset attacks due to linear ISN progression, 64 KB acceptance windows, and predictable polling rhythms. However, introducing an **In-Line Protective Gateway** with **Token Bucket Rate Limiting** and **Delayed Reset Quarantine (DRQ) with Active Zero-Window Challenge Probing** successfully neutralizes both high-speed micro-bursts and stealth low-rate attacks with zero modifications required to legacy RTU firmware.